

P0452r0

Binary-Binary transform_reduce(): The Missing Overload

Published Proposal, 14 October 2016

This version:

<http://wg21.link/P0452r0>

Author:

[Bryce Adelstein Lelbach](#) (Lawrence Berkeley National Laboratory)

Audience:

SG1, LEWG, LWG

Project:

ISO JTC1/SC22/WG21: Programming Language C++

Abstract

Renaming the parallel `inner_product()` overload to `transform_reduce()`, and unifying `transform_reduce()`'s interface with the rest of the algorithms library.

Table of Contents

- 1 `inner_product()` Cannot Be Parallelized
- 2 `inner_product()` is a Form of `transform_reduce()`
- 3 `transform_reduce()` is Missing an Overload
- 4 `transform_reduce()` Parameter Order is Odd
- 5 **Proposed Resolutions**
 - 5.1 Resolution 1: Rename `inner_product()` to `transform_reduce()` and Fix `transform_reduce()`
 - 5.2 Resolution 2: Weaken `inner_product()`
 - 5.3 Resolution 3: Weaken `inner_product()` and Fix `transform_reduce()`
 - 5.4 Resolution 4: Remove Parallel `inner_product()`
- 6 **Acknowledgement**
- References**

Informative References

While reviewing the C++17 CD and preparing [my CppCon talk](#), I found some issues revolving around `transform_reduce()` and the parallel form of `inner_product()`.

§ 1. `inner_product()` Cannot Be Parallelized

The parallel version of `inner_product()` (e.g. `ExecutionPolicy` overload) is not useful because the wording limits parallelization. 26.8.5 **[inner.product]** p1 of the C++17 CD states that `inner_product()`:

```
computes its result by initializing the accumulator acc with the initial value init and then modifying it with acc = acc + (*i1) * (*i2) or acc = binary_op1(acc, binary_op2(*i1,*i2))
```

Similar to `accumulate()`, while an `inner_product()` operation can be parallelized *in principle*, the wording for `inner_product()` prevents parallelization for reduction operations that are non-commutative and non-associative, like floating point addition. For such operations, the wording forces each iteration to depend on the prior iteration to ensure the correct ordering of accumulation. This introduces a loop-carried dependency that makes it impossible to parallelize.

§ 2. `inner_product()` is a Form of `transform_reduce()`

Just as `reduce()` is the parallel counterpart of `accumulate()` (same basic operation but without a specific ordering), `inner_product()` has a natural counter-part. Consider this possible implementation of `inner_product()`:

```
template <class InputIt1, class InputIt2,
          class T,
          class ReductionOperation, class BinaryTransformOperation>
T inner_product(InputIt1 first1, InputIt1 last1,
               InputIt2 first2, T init,
               ReductionOperation reduce_op,
               BinaryTransformOperation transform_op)
{
    while (first1 != last1)
    {
        init = reduce_op(init, transform_op(*first1, *first2));
        ++first1;
        ++first2;
    }
    return value;
}
```

The application of `transform_op` to the sequence is a binary `transform()`:

```
template <class InputIt1, class InputIt2,
          class OutputIt, class BinaryTransformOperation>
OutputIt transform(InputIt1 first1, InputIt1 last1, InputIt2 first2,
                  OutputIt o_first, BinaryTransformOperation transform_op)
{
    while (first1 != last1)
    {
        *o_first++ = transform_op(*first1++, *first2++);
    }
    return o_first;
}
```

And the application of `reduce_op` accumulates the transformed values:

```
template <class InputIt, class T, class ReductionOperation>
T accumulate(InputIt first, InputIt last, T init,
             ReductionOperation reduce_op)
{
    for (; first != last; ++first)
    {
        init = reduce_op(init, *first);
    }
    return init;
}
```

So, `inner_product()` is an `accumulate()`'s `reduce()`s of a `transform()`ed sequence. It's a `transform_reduce()`!.

§ 3. `transform_reduce()` is Missing an Overload

However, `transform_reduce()` is missing an overload for a binary transform; only overloads with unary transform operations are currently specified by the C++17 CD. I call this missing overload **binary-binary `transform_reduce()`**, since both the reduction and transformation operation are binary operations.

This overload, which is equivalent to parallel `inner_product()` with weaker ordering, is very useful. Many of the typical examples of `transform_reduce()` usage that I've seen use tricks to perform a binary-binary `transform_reduce()` using the unary-binary `transform_reduce()` that is in the C++17 CD.

The typical `transform_reduce()` dot product example (similar to what is found in the original `transform_reduce()` proposal [\[N3960\]](#)) looks like this:

```
std::vector<std::tuple<double, double> > XY = // ...

double dot_product = std::transform_reduce(
    std::par_unseq,

    // Input sequence.
    XY.begin(), XY.end(),

    // Unary transformation operation.
    [](std::tuple<double, double> const& xy)
    {
        // Array of struct means this is tricky to execute on vector hardware:
        //
        // memory layout: X[0] Y[0] X[1] Y[1] X[2] Y[2] X[3] Y[3] ...
        //
        // op #0: load a pack of Xs (they aren't contiguous; the load will be
        //                               strided, may need to access multiple
        //                               cache lines and may be harder for the
        //                               hardware prefetcher to handle)
        // op #1: load a pack of Ys (same as above)
        // op #2: multiply the pack of Xs by vector of Ys
        return std::get<0>(xy) * std::get(xy);
    },

    double(0.0), // Initial value for reduction.

    // Binary reduction operation.
    std::plus<double>()
);
```

Note that this is array-of-structs NOT struct-of-arrays. The HPX and THRUST dot product examples use iterator tricks (zip iterators or counting iterators) to switch to a struct-of-arrays scheme. The zip iterator trick is shown below, using Boost:

```

std::vector<double> X = // ...
std::vector<double> Y = // ...

double dot_product = std::transform_reduce(
    std::par_unseq,

    // Input sequence.
    boost::make_zip_iterator(X.begin(), Y.begin()),
    boost::make_zip_iterator(X.end(), Y.end()),

    // Unary transformation operation.
    [](auto&& xy) // std::tuple<double, double>-esque
    {
        // Struct of arrays means this is easier to execute on vector hardware:
        //
        // memory layout: X[0] X[1] X[2] X[3] ... Y[0] Y[1] Y[2] Y[3] ...
        //
        // op #0: load a pack of Xs (elements are contiguous, load will access
        //                               only one cache line and the hardware
        //                               prefetcher will easily track the memory
        //                               stream)
        // op #1: load a pack of Ys (same as above)
        // op #2: multiply the pack of Xs by the pack of Ys
        return std::get<0>(xy) * std::get<1>(xy);
    },

    double(0.0), // Initial value for reduction.

    // Binary reduction operation.
    std::plus<double>()
);

```

More examples of this pattern can be found in HPX ([zip iterator example](#) and [counting iterator example](#)) and THRUST ([zip iterator example](#)).

§ 4. `transform_reduce()` Parameter Order is Odd

The missing binary-binary `transform_reduce()` would look like this:

```

template <class InputIt1, class InputIt2,
          class BinaryTransformOperation,
          class T,
          class ReductionOperation> // Always binary.
T transform_reduce(InputIt1 first1, InputIt1 last1,
                  InputIt2 first2,
                  BinaryTransformOperation transform_op,
                  T init,
                  ReductionOperation reduce_op);

```

Note that the order of parameters, which is consistent with the other `transform_reduce()` overloads, is inconsistent with `inner_product()`, `transform()` and `reduce()`:

```

template <class InputIt1, class InputIt2,
         class T,
         class ReductionOperation, class BinaryTransformOperation>
T inner_product(InputIt1 first1, InputIt1 last1,
               InputIt2 first2, T init,
               ReductionOperation reduce_op,
               BinaryTransformOperation transform_op)

template <class InputIt1, class InputIt2,
         class OutputIt, class BinaryTransformOperation>
OutputIt transform(InputIt1 first1, InputIt1 last1, InputIt2 first2,
                  OutputIt o_first, BinaryTransformOperation transform_op);

template <class InputIt, class T, class ReductionOperation>
T accumulate(InputIt first, InputIt last, T init,
             ReductionOperation reduce_op);

```

The inconsistencies are:

- In `inner_product()`, `accumulate()` and `reduce()` (which is not shown but has the same interface as `accumulate()`), the initial value `init` parameter comes after the iterator parameters and before the operation parameters. In `transform_reduce()`, it is in between the transformation operation parameter and the reduction operation parameter.
- In `inner_product()`, the reduction operation parameter comes before the transformation operation parameter.
- `inner_product()`, `accumulate()` and `reduce()` have overloads which do not take any operations and use `operator+` for reduction and `operator*` for transformation.

§ 5. Proposed Resolutions

I filed a US national body ballot comment about the issue with the parallel `inner_product()` overload. The following solutions would resolve that comment.

§ 5.1. Resolution 1: Rename `inner_product()` to `transform_reduce()` and Fix `transform_reduce()`

Rename the useless parallel `inner_product()` with a binary-binary `transform_reduce()`, and we should change the `transform_reduce()` interface to have the same parameter order as `inner_product()`, `accumulate()`, `reduce()` and `transform()`.

Specifically, we should:

- Rename the `ExecutionPolicy` overload for `inner_product()` to `transform_reduce()`.
- Add a new serial binary-binary `transform_reduce()` overload.
- Add two new binary-binary `transform_reduce()` overloads (one serial overload and one parallel `ExecutionPolicy` overload).
- Change the parameter order for all `transform_reduce()` overloads so that they are consistent with `inner_product()`, `accumulate()`, `reduce()` and `transform()`:
 - The initial value parameter comes after the iterator parameters and before the operation parameters.
 - The reduction operation parameter comes before the transformation operation parameters.
- Add `transform_reduce()` overloads that do not take operation parameters and use `operator+` for reduction and `operator*` for transformation.

With these changes, a parallel dot product over struct-of-arrays data can be written as:

```
std::vector<double> X = // ...
std::vector<double> Y = // ...

double dot_product =
    std::transform_reduce(std::par_unseq, x.begin(), x.end(), y.begin(), double(0.0));
```

A parallel word count could be written with binary-binary `transform_reduce()` as:

```
bool is_word_beginning(char left, char right)
{
    return std::isspace(left) && !std::isspace(right);
}

std::size_t word_count(std::string_view s)
{
    if (s.empty()) return 0;

    // Count the number of characters that start a new word.
    return std::transform_reduce(
        std::par_unseq,

        // "Left" input sequence: s[0], s[1], ..., s[s.size() - 2]
        s.begin(), s.end() - 1,
        // "Right" input sequence: s[1], s[2], ..., s[s.size() - 1]
        s.begin() + 1,

        // Initial value for reduction: if the first character
        // is not a space, then it's the beginning of a word.
        std::size_t(!std::isspace(s.front()) ? 1 : 0),

        // Binary transformation operation.
        std::plus<std::size_t>(),

        // Binary transformation operation: Return 1 when we hit
        // a new word.
        is_word_beginning
    );
}
```

§ 5.2. Resolution 2: Weaken `inner_product()`

Alternatively, we could weaken ordering required by `inner_product()`, which would make it equivalent to binary-binary `transform_reduce()`. This approach is not backwards compatible; implementations could rewrite their serial `inner_product()` in a way that uses a different ordering than the one previously required, breaking user code. Additionally, different implementations might provide different orderings, which could cause portability issues.

However, this would still leave `transform_reduce()` with an interface that is inconsistent with other algorithms that implement very similar behavior. If we ship `transform_reduce()` with a broken interface, it will be difficult to fix it later because we would have to break users code.

§ 5.3. Resolution 3: Weaken `inner_product()` and Fix `transform_reduce()`

Adopt Resolution 2, and also the parts of Resolution 1 that fix the inconsistencies in `transform_reduce()`:

- Change the parameter order for all `transform_reduce()` overloads so that they are consistent with `inner_product()`, `accumulate()`, `reduce()` and `transform()`:
 - The initial value parameter comes after the iterator parameters and before the operation parameters.

- The reduction operation parameter comes before the transformation operation parameters.
- Add `transform_reduce()` overloads that do not take operation parameters and use `operator+` for reduction and `operator*` for transformation.

§ 5.4. Resolution 4: Remove Parallel `inner_product()`

The simplest approach, and least desirable, is to simply remove the parallel `inner_product()` overloads. This removes useful functionality which was intended to go into the standard (e.g. binary `transform_reduce()`). Additionally, it would leave `transform_reduce()`'s interface inconsistent with the other algorithms, and it will be difficult to fix that interface in the future because it would be a breaking change.

§ 6. Acknowledgement

Thanks to:

- Agustín Bergé for helping identify this issue.
- Hartmut Kaiser, JF Bastien, Michael Garland and Jared Hoberock for providing feedback.

§ References

§ Informative References

[N3960]

Jared Hoberock. [Working Draft, Technical Specification for C++ Extensions for Parallelism](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2014/n3960.pdf). 28 February 2014. URL: <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2014/n3960.pdf>